Time left 1:39:45

Question 1
Not yet answered
Marked out of 1.00
Flag question

Warmup

1. Download the archive with the HAL9000 code with usermode working.

2. In your HAL9000 folder rename the **src** folder to **src_old**. Copy the **src** from the archive to alongside **src_old**. You'll need to have Visual Studio closed for this.

3. Open HAL9000.sln in Visual Studio, choose the **Threads** configuration and **Rebuild** the HAL9000 solution.

4. Rebuild the **RemoveAllTests** project because we will not run any pre-defined test during the exam.

A. Threads

1. (2p) Modify the way in which thread ID allocation works, such that the TIDs are allocated according to the following rule:

- TID = Prev_TID - 0x31300
- Initial TID should be 0xFFFF'FFFF => The threads should have 0xFFFF'FFFF, FFFC'ECFF, FFF9'D9FF, etc...

Display at thread creation the message "Thread [tid=0x%X] is being created".

2. (4p) Maintain the number of living children and a pointer to the parent for each thread. These values should be updated each time a thread is created or destroyed. Whenever a thread is destroyed, the children of that thread should be added to the parent. When a thread is created, log the following information:
"Thread [%s] is being created and it has %d living siblings."

For example, if the threads are created in the order they are mentioned:

"thread-1" creates:
"thread-2", which creates:
"thread-3"
"thread-4"
"thread-5"

The output should be the following:

"Thread [thread-1] is being created and it has 0 living siblings."
"Thread [thread-2] is being created and it has 0 living siblings."
"Thread [thread-3] is being created and it has 0 living siblings."
"Thread [thread-4] is being created and it has 1 living siblings."
"Thread [thread-5] is being created and it has 1 living siblings."

Hint: In order to completely implement the solution, you may need to keep the list of children for each thread

3. (0.5p) Modify the "/threads" command to print the number of children and the parent TID for each thread, in addition to the fields it already displays.

4. (2.5p) Implement the "/testthreadinfo" command which performs the following:
Creates 2 threads (T1, T2). T1 creates 3 threads (T1-1, T1-2, T1-3). T2 creates 2 threads (T2-1, T2-2).

Note: No thread should terminate before its children's termination

B. User prog + Virtual Memory

1. (2.5p) Make the changes required to run user applications

- Setting up the user stack
- Implement *SyscallValidateInterface* (if not implemented already)
- Implement *SyscallProcessExit* and *SyscallThreadExit*
- Implement the write to UM_FILE_HANDLE_STDOUT system call

2. (2p) Modify the exception handler such that an unresolved user-mode exception will terminate the currently executing process. Print the name of the process, the index of the exception, the name of the exception and the instruction pointer before terminating the process. The message should have the format *"Exception 0x%X(exception name: %s) at address 0x%X will cause the termination of process %s!\n"*. You can test this part by running */procstart C:\Applications\DUMMY.EXE* and */proctest BadWriteNull*

Hint: Use *GdtIsSegmentPrivileged* in order to determine the privilege level associated to the current code segment

Hint2: You can check the way function *DumpProcessorState* logs the value of the instruction pointer.

3. (3.5p) Implement a new system call:

```
STATUS
SyscallGetChildrenCount(
  IN  TID                 ThreadId,
  OUT QWORD*              ChildrenCount
);
```

The function should return the number of children of the thread specified by ThreadId. The thread can be determined by iterating through the list of all the threads in the system (AllThreadsList). If the ThreadId is not present in the list, it should return STATUS_UNSUCCESSFUL. If you did not implement the count for the children, return any number.

Hint: Don't forget to have synchronised access to the global AllThreadsList.

4. (1p) Test the system call SyscallGetChildrenCount by modifying a user application (for example Dummy.exe) to call the system call.

Implement SyscallThreadGetTid without parameter validation, to get the current thread's TID ignoring the ThreadHandle parameter. This way SyscallThreadGetTid should always return the TID of the current thread, which you can give as an argument to SyscallGetChildrenCount.

Hint: Use "/procstart C:\Applications\Dummy.exe" to run the application.

5. (3p) Implement the syscalls SyscallVirtualAlloc and SyscallVirtualFree system call (ignoring the FileHandle and Key parameters for the first one).

Also, take into account the fact that each process should have a quota of **maximum 10 commited pages at a given time**. If a process tries to alloc more than its quota, it should receive an error status.

NOTES:
-> Even if not explicitly mentioned, you should not forget we are working in a multi-threading system, synchronization is always necessary when accessing shared structures.
-> Even if not explicitly mentioned, you should also validate user input when implementing system calls.

Evaluation criteria:
MANDATORY:

- Compilation with no errors in neither configurations: "Threads", "Userprog" and "VirtualMemory".
- HAL9000 Boots up (/help command can be executed with success).

A. Threads

1. 2p
1p: TID sequence implemented (maintaining atomicity).
1p: Print at thread creation (for all threads, including the first main threads of the CPUs).

2. 4p
0.5p: Adding the fields
1p: Initializing parent and children fields
0.5p: Updating the children value at thread creation
1p: Updating the children and parent value at thread termination
1p: Log information at thread creation

3. 0.5p
0.25p: Log the number of children
0.25p: Log the parent TID

4. 2.5p
1p: Correctly add command
1p: Correctly create thread hierarchy (0.5p for each layer 1, 0.5p for layer 2)
0.5p: The correct information is logged

B. Userprog + Virtual Memory

1. 2.5p
0.5p: User stack setup
0.5p: Correct implementation for SyscallProcessExit
0.5p: Correct implementation for SyscallThreadExit
1p: Correct implementation for SyscallFileWrite

2. 2p
0.5p: Correct implementation of exception handling if the process is usermode
0.5p: Determining the exception code
0.5p: Determining the exception name
0.5p: Determining the instruction pointer

3. 3.5p
0.5p: Adding the syscall identifier
1p: Adding the user-mode side
1p: Adding the syscall handler
1p: Syscall functionality correctly implemented

4. 1p
0.5p: Implement SyscallThreadGetTid
0.5p: Calling the system call from user-mode

5. 3p
1p: Implement SyscallVirtualAlloc
1p: Implement SyscallVirtualFree
0.5p: Maximum allocation per process
0.5p: Test in usermode application

LAB_GRADE = Threads 9p + (Userprog + Virtual Memory) 12p = 21p max

MANDATORY: Grade(Threads) >= 4.5 && Grade(Userprog + Virtual Memory) >= 6.

Maximum file size: 2MB, maximum number of files: 1

Files

You can drag and drop files here to add them.

Finish attempt ...

Quiz navigation

1

Finish attempt ...

PREVIOUS ACTIVITY
Lab examination context and rules

NEXT ACTIVITY
Submit solutions to lab exam on January, 2023 (only if not submitted on the quiz)

Jump to...